BLA_02 · SOFTWARE ENGINEERING

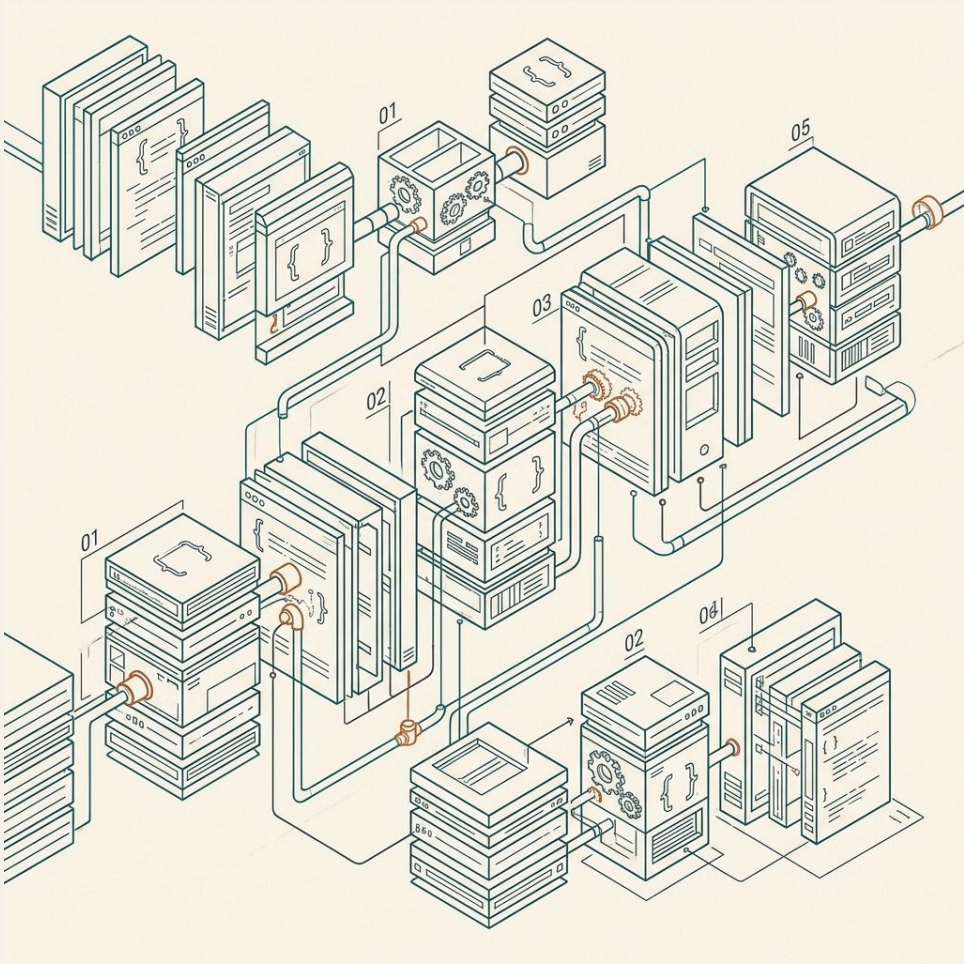
Software Process Models

From Waterfall to agile, and how Netflix and Riot Games choose their processes.

Truong Phat Tu

Master of Computer Science in Concordia University Chicago

Models: Waterfall · Incremental · Integration and configuration · Plan-driven · Agile
Cases: Netflix streaming platform · League of Legends



PART 1

What is a software process?

DEFINITION

A structured set of activities required to develop a software system, together with the order in which they happen.

Typical activities include specifying a data model or designing a user interface.

> The key is quality control, process control, and management of production, cost and human resources.

WHY AN ORGANIZED PROCESS, NOT ONLY PROGRAMMING?

Products

The outcomes of a process activity.

Roles

The responsibilities of the people involved in the process.

Pre- and post-conditions

Statements that are true before and after an activity is carried out or a product is produced.

Four fundamental process activities

01

Specification

Defines what the system should do and the constraints it works within.

- Feasibility study
- Elicitation and analysis
- Requirements specification
- Requirements validation

02

Design and implementation

Turns the specification into a working system.

- Architecture, components, interfaces
- Data structures and algorithms
- Coding, debugging, integrating

03

Validation

Shows the system conforms to its specification and meets real customer needs.

- Reviews and inspections
- Unit, system and acceptance testing

04

Evolution

Modifies the system after delivery as business needs, platforms and user requests change.

- Corrective, adaptive, perfective
- Reuses the existing system

Specification feeds development, development produces a system for validation, and validation results feed back into earlier work. Once delivered, evolution restarts the cycle with new or changed requirements.

PART 1

Specification and validation in detail

> Mistakes in specification are the most expensive to fix later, which is why this stage matters so much.

SPECIFICATION INCLUDES

Feasibility study

Is the system technically and financially worth building?

Elicitation and analysis

Interviews, observation or prototypes to find stakeholder needs.

Specification

User requirements (high level) and system requirements (detailed).

Requirements validation

Check they are realistic, consistent and complete.

Verification

Are we building the product right?

Checks the system against its specification through reviews and inspections of requirements, design and code.

Validation

Are we building the right product?

Checks the system meets the customer's real needs, including acceptance testing with the customer.

Testing levels

Unit, system, acceptance

Testing is the most common technique, but it can only show that defects are present, not that none remain.

PART 1

Example: QA automation in a project

STEP 1



Write the tests

Tests use a framework such as pytest with a browser automation tool to simulate user interactions.

STEP 2



Run from the command line

Developers run the automated tests in their local development environment.

STEP 3



Run in CI/CD

The same tests run in the project's CI/CD pipeline on every change.

STEP 4



Catch regressions

Failures are caught before the code is deployed to users.

WHERE THE FOUR ACTIVITIES APPEAR

Specification

Expected user behaviour becomes test cases.

Development

Application code and test code are written together.

Validation

Automated runs verify the application behaves as specified.

Evolution

Tests rerun on each change to protect existing behaviour.

PART 2

Software process models

A process model is a **simplified description** of a software process, presented from one perspective.

Plan-driven

All process activities are planned in advance, and progress is measured against that plan.

Agile

Planning is incremental, so it is easier to change the process to reflect changing customer requirements.

> In practice, most processes include elements of both. There are no right or wrong software processes.

PART 2

Five approaches compared

Approach	Basic idea	How work progresses	Suited to
Waterfall	Sequential, linear phases: requirements, design, implementation, testing, maintenance.	Step by step; each stage is completed before the next begins.	Large projects with fixed, well-understood requirements, strict regulation or safety needs (aerospace, medical, infrastructure).
Incremental	Build small pieces and deliver functional components early.	Repeating cycles add features to a growing baseline version.	Evolving requirements and a need for early release or feedback (web apps, SaaS tools).
Integration and configuration	Assemble systems from off-the-shelf software, commercial packages or open-source components.	Evaluate, select, adapt and connect existing components.	Tight deadlines or budgets and standard business systems (CRM, ERP).
Plan-driven	Plan every development activity in detail before implementation begins.	Pre-scheduled milestones and formal documentation tracked against a master plan.	Long-term enterprise work, fixed-price contracts, multi-team coordination, formal certification.
Agile	Iterative, human-centered delivery with customer collaboration and fast response to change.	Short time-boxed sprints that each produce a working increment.	Startups, innovative products and customer-facing web or mobile apps.

PART 3

The Waterfall model

01

Requirements analysis

Gather and document all functional and non-functional needs up front.

02

System and software design

Define architecture, data models and component structures.

03

Implementation and unit testing

Code each module and verify its basic functionality.

04

Integration and system testing

Combine modules into a full system and test it as a whole.

05

Deployment and maintenance

Deliver to production, fix remaining bugs, handle long-term updates.

WHY WATERFALL IS PLAN-DRIVEN

Rigid plan first. Development follows a pre-scheduled plan set before any code is written.

Tracked strictly. Every milestone, deliverable and resource allocation is mapped out upfront.

PART 3

Documentation, phase gates and late change

Documentation

Primary communication medium
Documents act as the contract and reference across sequential teams.

Basis for sign-off
Each phase hands the next a complete, written deliverable.

Phase completion

Reviewed and signed off
Each stage is fully reviewed before the next begins.

Stops defects cascading
Problems are caught at the gate, not passed downstream.

When requirements change late

High cost and delay
Teams must re-evaluate earlier phases, re-architect components and rewrite code.

Scope and schedule disruption
Earlier documentation is invalidated, deadlines slip and total cost grows.

PART 4

Incremental development

Break the system into pieces

Functionality is built through successive releases; each delivers a usable subset.

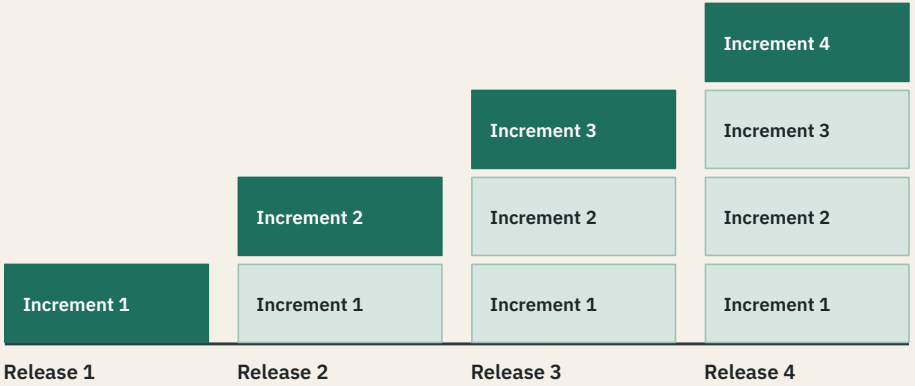
Interleave the activities

The team specifies the next feature while developing current ones and validating finished ones.

Customer feedback steers it

Customers evaluate working software after each release; feedback shapes and prioritizes later increments.

GROWING BASELINE ACROSS RELEASES



Customer feedback after each release shapes the next increment.

Why increments lower the cost of change

Small increments keep change local.

Incremental delivery also puts useful functionality in front of customers earlier.

Small, isolated pieces

Changes touch a small piece of code rather than an entire monolithic system.

Less documentation rework

Little time is wasted revising extensive documents for work already done.

No re-architecting of phases

Completed work is extended instead of rebuilt in earlier phases.

PART 5

Integration and configuration

Build the system by reusing existing software, components, frameworks, services or configurable systems instead of writing everything from scratch.

01

Requirements specification

Outline the requirements without fixing every detail.

02

Discovery and evaluation

Find candidate software and judge how well it fits.

03

Requirements refinement

Adjust requirements to what the discovered software offers.

04

Configuration and adaptation

Configure the chosen system or adapt components to fit.

05

Integration

Connect the components and services into one system.

Advantage

Faster delivery at lower cost and risk, because proven software is reused.

Limitation

Requirements may need compromise, and the team has less control over how reused components evolve.

PART 6

Evolution: engineering continues after delivery

60%+

of total software cost goes to maintenance

Figure from the lab notes. Software production is development plus maintenance.

THREE KINDS OF EVOLUTION

Corrective

Fixing faults found after delivery.

Adaptive

Adjusting to a changed environment such as a new operating system or hardware.

Perfective

Adding or improving functions and performance.

PART 7

Coping with change



Change is inevitable

In all large software projects

Business changes lead to new and changed requirements.

New technologies open up better ways to implement the system.

Changing platforms require application changes.

Change causes **rework** plus the cost of new functionality.



Prototyping

Supports change anticipation

A version of the system, or part of it, is built quickly to check requirements and design feasibility.

Requirements: elicitation and validation.

Design: explore options and develop the UI.

Testing: run back-to-back tests.



Incremental delivery

Supports change avoidance and tolerance

Increments go to the customer for comment and experimentation.

Customers receive useful functionality earlier.

Feedback from each increment informs what is built next.

PART 7

How change affects a project

Cost

Rework such as **re-analysing requirements**, plus the cost of implementing the new functionality.

Schedule

Rework and re-planning **push back delivery dates** and can disturb milestones.

Testing

Changed code must be **retested**, with regression checks to catch side effects.

Documentation

Requirements, design and user documents must be **updated to stay consistent** with the system.

Two real-world systems

STREAMING PLATFORM · NETFLIX

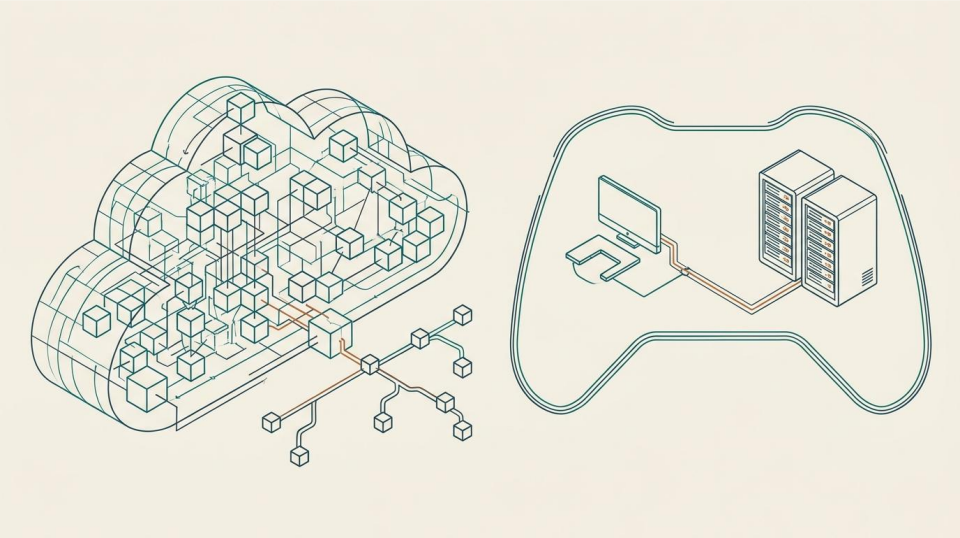
Netflix streaming service

A cloud-based multimedia platform that delivers continuous high-definition video on demand to thousands of device types without interruption.

GAMING SYSTEM · RIOT GAMES

League of Legends

A real-time, low-latency client-server platform that synchronizes player inputs, physics and rendering across multiplayer matches worldwide.



PART 8

Core technical functions

Netflix

Transcoding and encoding

Converts raw video into many formats and bitrates for different screens and speeds.

Content delivery network

Caches popular video on edge servers near users to cut delay and buffering.

Recommendation engine

Analyzes real-time behavior to personalize content suggestions.

Adaptive bitrate streaming

Adjusts resolution mid-stream to match bandwidth and prevent buffering.

League of Legends

Game client and engine

Handles 3D rendering, animation, input and local sound on the player’s device.

Dedicated game servers

Single source of truth for match state; run game logic to prevent cheating.

Low-latency networking

Optimized UDP protocols process player actions in milliseconds.

Matchmaking and lobby

Groups players of similar skill into balanced lobbies before a match starts.

PART 9

Netflix and Riot Games side by side

Dimension	Netflix (streaming platform)	Riot Games · League of Legends
Process orientation	Fully continuous, microservice-driven DevOps	Scheduled bi-weekly incremental release sprint cycles
Deployment cadence	Continuous: thousands of microservice pushes per day	Structured: bi-weekly global game client patches
Testing focus	Resiliency testing, chaos engineering and automated canary analysis	Balance verification, public beta playtesting (PBE) and client-server integration
Handling changes	Feature toggles and dynamic A/B routing	Incremental patches, balance hotfixes and seasonal overhauls
User feedback loop	Automated telemetry and live analytics	Community feedback, win-rate telemetry and PBE testing

Similar: both ship incrementally, rely on telemetry and automate testing. **Different:** Netflix deploys continuously, while Riot releases on a fixed two-week cadence.

PART 9

Process profile of each system

Aspect	Netflix	Riot Games · League of Legends
Development approach	Microservices-based DevOps, CI/CD and agile incremental development on AWS	Hybrid agile (Scrum/Kanban) with scheduled patch-release increments
Requirements	Data-driven user stories from telemetry, A/B test metrics and feature flags	Balance data analysis, community feedback, backlog and design documents
Waterfall traits	Low: upfront planning mainly for cloud foundations and compliance baselines	Moderate: art pipelines, core engine milestones and tournament build freezes
Agile traits	High: continuous iteration, short feedback loops, rapid deployment	High: gameplay balancing, seasonal updates and rapid live hotfixes
Major advantage	High availability, fault isolation and global rollouts without interruption	Predictable update cadence with flexibility to rebalance as the meta shifts
Major challenge	Complex service orchestration, latency and distributed consistency	Creative freedom versus engine stability, regressions and legacy code debt

PART 10

Waterfall: advantages and disadvantages

Advantages

Clear structure and documentation

Comprehensive upfront specifications make scope, requirements and deliverables easy to track.

Predictable planning

Timelines, milestones and budgets can be fixed before development begins.

Ease of management

Phase gates with formal sign-offs give clear accountability and measurable progress.

Disadvantages

Inflexible to change

Changing requirements mid-stream is costly once early phases are locked in.

Late validation

Working software arrives late, delaying user testing and early bug discovery.

High upfront risk

Requirement misunderstandings can stay hidden until system testing or deployment.

PART 10

Choosing the right approach

Waterfall fits when

- Requirements are fixed, stable and fully understood.
- Regulatory, safety or legal standards apply (aerospace, medical devices, defense).
- Hardware and software are co-designed and hardware cannot change after production.

Incremental or agile fits when

- Requirements evolve or are uncertain.
- Speed to market matters for web, mobile or cloud applications.
- Rapid user feedback and continuous updates drive success.

WHY LARGE PROJECTS COMBINE MODELS

Risk across subsystems

Waterfall for hardware or compliance modules; agile for user-facing interfaces.

Scale and coordination

Upfront architecture gives many teams a baseline; each team iterates in short sprints.

Flexibility with governance

Management tracks predictable milestones while engineers adapt during implementation.

PART 11

Why successful companies differ (1 of 2)

Factor	Netflix	Riot Games
Type of software	Thousands of loosely coupled cloud microservices; a change to one rarely breaks the app, so decoupled DevOps pipelines work.	A heavy local client plus low-latency multiplayer servers; core engine changes need tight client-server synchronization.
Team size	Thousands of engineers in hundreds of autonomous teams, each owning a service end to end.	Large cross-functional teams of engineers, artists, sound designers, QA and balance designers aligning on each patch.
Safety and security	Operational resilience and fault tolerance, with automated chaos testing in live environments.	Competitive integrity: anti-cheat and server-side validation, since sync bugs can enable exploits.
Frequency of change	Continuous, driven by live telemetry, streaming quality metrics and A/B tests.	Rapid, but channelled into predictable bi-weekly patches so players and leagues can adapt.

PART 11

Why successful companies differ (2 of 2)

Factor	Netflix	Riot Games
Customer expectations	24/7 uninterrupted playback, instant loading and seamless updates with no downtime.	Predictable patch cadence, polished content, detailed patch notes and notice of maintenance.
Regulation	Digital rights management, content licensing, GDPR and local streaming rules.	Age ratings (ESRB, PEGI), loot-box disclosure laws and esports governance.
Cost and risk	Downtime or degraded video is the biggest risk, so features ship behind flags with canary deployments.	A broken patch can crash clients or ruin competitive matches, so the PBE validates changes first.
Release frequency	Hundreds to thousands of production deployments every day.	A global patch every two weeks, plus instant hotfixes for urgent bugs.

PART 11

No single process fits every project

A process succeeds when it matches the system's architecture, user expectations and organizational goals.

Netflix

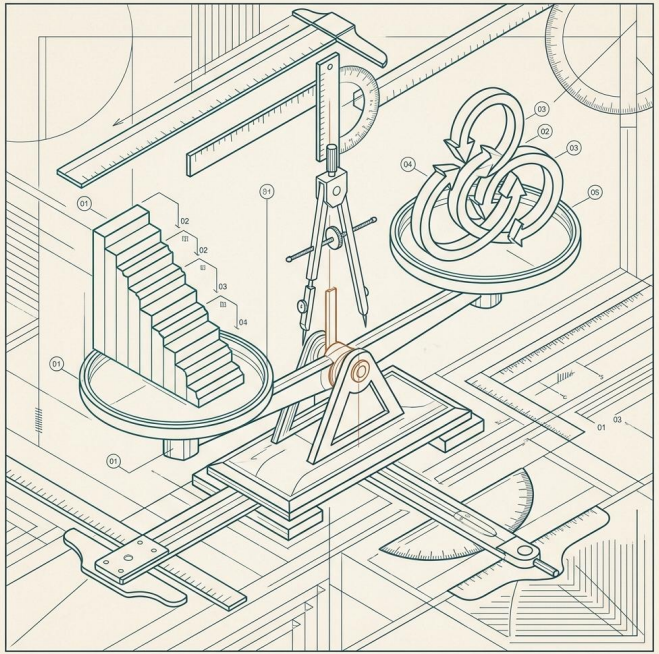
Cloud-native microservices thrive on continuous micro-deployments.

Riot Games

A competitive game thrives on structured, incremental releases.

Your project

Weigh software type, team size, safety, change, customers, regulation, cost and release frequency.



SOURCES

References

01 Royce, W. W. (1970). Managing the development of large software systems. Proceedings of IEEE WESCON, 26, 328–338.

02 Sommerville, I. (2015). Software Engineering (10th ed.). Pearson. Foundational textbook for the Waterfall model, incremental development, interleaved specification and development, and plan-driven versus agile methods.

03 Mokhtar, R., and Khayyat, M. (2022). A comparative case study of waterfall and agile management. SAR Journal – Science and Research, 52–62. doi.org/10.18421/sar51-07

04 F5 NGINX Developer Blog (2015). Adopting microservices at Netflix: Lessons for architectural design. F5 Networks.

05 Riot Games Engineering (2016–2021). The Riot Games API: Deep Dive, and Keeping Legacy Software Alive: A Case Study. Riot Games official news and tech articles.